

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

**Đề tài: Finetune mô hình dịch máy Neural cho
văn bản chuyên ngành y khoa Anh-Việt**

Lớp tín chỉ:	D23CQCN04-B
Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Đức Trung
Mã sinh viên:	B23DCCN858

Hà Nội 2026

MỤC LỤC

TÓM TẮT	3
PHẦN 1: HỆ THỐNG MEDICAL RAG CHATBOT	3
1.1 Tổng quan hệ thống	3
1.2 Kiến trúc hệ thống.....	3
1.3 Cơ sở tri thức y học.....	4
1.4 Tính năng nổi bật 1: Tìm kiếm kết hợp Hybrid Retrieval	4
1.5 Tính năng nổi bật 2: Hỗ trợ đa ngôn ngữ đầu vào (Language Detection & Translation) .	5
1.7 Luồng xử lý RAG Pipeline (End-to-End)	5
1.8 Công nghệ sử dụng	6
PHẦN 2: THAY ĐỔI VÀ MỞ RỘNG TRONG DỰ ÁN CŨ (MedTrans)	6
2.1 Thay đổi cơ sở dữ liệu.....	6
2.2 Tích hợp API Chatbot vào Backend (Spring Boot)	7
2.3 Thay đổi giao diện (Frontend - React).....	7
PHẦN 3: ĐƠN GIẢN HÓA BẢO MẬT.....	8
3.1 Các thay đổi về bảo mật.....	8
3.2 Chi tiết các thay đổi	8

TÓM TẮT

Tuần này, dự án được mở rộng từ hệ thống dịch thuật MedTrans bằng việc xây dựng thêm module Chatbot Y tế (Medical RAG). Module mới này hoạt động song song với chức năng dịch thuật hiện có, cho phép người dùng tương tác với chatbot chuyên về các vấn đề y khoa. Chatbot áp dụng kỹ thuật tìm kiếm kết hợp Hybrid Retrieval (ChromaDB + BM25) kết hợp với các mô hình ngôn ngữ lớn (Google Gemini / Qwen) để tạo ra các câu trả lời có căn cứ từ cơ sở tri thức y học đáng tin cậy.

PHẦN 1: HỆ THỐNG MEDICAL RAG CHATBOT

1.1 Tổng quan hệ thống

Hệ thống Medical RAG (Retrieval-Augmented Generation) là một microservice độc lập viết bằng Python (FastAPI), được tích hợp vào ứng dụng MedTrans thông qua REST API. Hệ thống cung cấp khả năng trả lời các câu hỏi y tế bằng tiếng Việt và tiếng Anh, dựa trên cơ sở tri thức y học được lưu trữ nội bộ.

1.2 Kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình Kiến trúc Sạch (Clean Architecture), phân tách rõ ràng các tầng chức năng:

Tầng / Layer	Thành phần	Chức năng
API Layer	api/v1/chat.py, api/dependencies.py	Tiếp nhận HTTP request, định tuyến, quản lý Dependency Injection (Singleton Services)
Core Layer	core/config.py, core/exceptions.py, core/logging.py	Cấu hình hệ thống từ biến môi trường .env, xử lý lỗi, cấu hình log toàn cục
Domain Layer	domain/interfaces.py, domain/models.py	Định nghĩa các Interfaces/Contracts và Pydantic Models cho dữ liệu vào/ra
Infrastructure Layer	infrastructure/embeddings.py, retrievers.py, vectorstore.py, translator.py, llm/	Triển khai kết nối với ChromaDB, mô hình BGE-M3, BM25, EnViT5, Gemini/Qwen
Services Layer	services/rag_pipeline.py	Điều phối toàn bộ vòng đời xử lý RAG
Prompts	prompts/medical_prompt.py	Quản lý các mẫu prompt y tế (Tiếng Việt và Tiếng Anh)

1.3 Cơ sở tri thức y học

a) Cơ sở dữ liệu hiện tại

Cơ sở dữ liệu lưu trữ tại thư mục `data/raw/`, được phân chia theo bệnh lý:

- Cúm (Influenza) - Từ tài liệu Vinmec và các nguồn y tế uy tín
- Hen suyễn (Asthma)
- Thiếu máu (Anemia)
- Tiểu đường (Diabetes)
- Tăng huyết áp (Hypertension)

Tài liệu được lưu trữ định dạng Markdown, sau đó được nạp vào ChromaDB (dense vector store) và xây dựng chỉ mục BM25 (sparse index) thông qua script `ingest_documents.py`.

b) Quy trình thu thập dữ liệu tự động (Crawl dữ liệu)

Để xây dựng được cơ sở tri thức y học đáng tin cậy nêu trên, dự án sử dụng một công cụ thu thập dữ liệu tự động được cấu trúc hoàn chỉnh trong file mã nguồn `crawl.py`. Quy trình crawl dữ liệu hoạt động theo hai chiến lược linh hoạt:

- **Chiến lược 1 (Duyệt theo danh mục bệnh):** Công cụ thực hiện quét qua trang danh mục bệnh lý chính thức tại đường dẫn <https://www.vinmec.com/vie/benh/>. Script sẽ tự động phân tích cấu trúc HTML, bóc tách các liên kết của từng bài viết cụ thể dựa trên biểu thức chính quy (Regex) và thực hiện phân trang tự động bằng cách thay đổi tham số `?page=N`. Phương pháp này giúp tránh được cơ chế chặn (Cloudflare) và kiểm soát tốc độ tải dữ liệu an toàn.
- **Chiến lược 2 (Tìm kiếm theo từ khóa cụ thể):** Khi kích hoạt chế độ `SEARCH_MODE = True` trong file cấu hình, mã nguồn sẽ sử dụng thư viện Selenium để khởi tạo trình duyệt ảo (Chrome Driver). Script tự động thực hiện truy vấn tìm kiếm bằng từ khóa (ví dụ: *"thiếu máu là gì"*), điều khiển trình duyệt giả lập hành vi người dùng bằng JavaScript để nhấn vào tab **"Bệnh"**, bóc tách toàn bộ danh sách liên kết trả về từ kết quả tìm kiếm và kích hoạt các nút "Xem thêm" hoặc "Trang kế tiếp" để gom tối đa lượng dữ liệu cần thiết.

c) Quá trình trích xuất nội dung bài bệnh

Sau khi danh sách liên kết được tổng hợp, file `crawl.py` sẽ duyệt qua từng URL để trích xuất nội dung theo các bước nghiêm ngặt:

- **Lọc sạch dữ liệu nhiễu (Denoising):** Tự động loại bỏ các thành phần HTML không liên quan đến nội dung y khoa như: thanh điều hướng (nav), chân trang (footer), mã script, các biểu ngữ quảng cáo (banner), widget và thông tin bác sĩ.
- **Cấu trúc hóa văn bản (Structuring):** Phân tách rõ ràng tiêu đề chính (`<h1>`), mô tả ngắn từ thẻ meta dữ liệu, các tiêu đề mục (`<h2>`, `<h3>`), các đoạn văn (`<p>`), danh sách liệt kê (`<ul>`, `<ol>`) và định dạng lại các bảng số liệu (`<table>`) thành dạng văn bản thô dễ xử lý
- **Lưu trữ:** Làm sạch tên file bằng cách loại bỏ các ký tự đặc biệt gây lỗi hệ thống tệp và lưu lại dưới dạng file `.md`

1.4 Tính năng nổi bật 1: Tìm kiếm kết hợp Hybrid Retrieval

Đây là tính năng kỹ thuật cốt lõi của hệ thống, kết hợp hai phương pháp tìm kiếm để tối đa hóa độ chính xác và độ bao phủ:

Phương pháp	Công nghệ	Ưu điểm
Dense Vector Search	ChromaDB + mô hình nhúng BGE-M3 (BAAI/bge-m3)	Tìm kiếm theo ngữ nghĩa (semantic), hiểu được ý nghĩa câu hỏi dù dùng từ khác nhau
Sparse Vector Search	BM25 (chỉ mục lưu tại data/bm25/bm25_index.pkl)	Khớp chính xác từ khóa, hiệu quả với thuật ngữ chuyên ngành y tế
Hybrid RRF Fusion	Reciprocal Rank Fusion (RRF)	Kết hợp kết quả của cả hai phương pháp trên, trích xuất tối đa 4 đoạn tài liệu phù hợp nhất

Luồng xử lý Hybrid Retrieval:

1. Nhận câu hỏi đầu vào từ người dùng (tiếng Việt hoặc tiếng Anh).
2. Nếu câu hỏi là tiếng Anh: dịch sang tiếng Việt bằng mô hình EnViT5 trước khi truy vấn.
3. Thực hiện đồng thời Dense Search trên ChromaDB và Sparse Search trên chỉ mục BM25.
4. Áp dụng thuật toán RRF Fusion để hòa trộn và xếp hạng kết quả từ cả hai nguồn.
5. Trích xuất tối đa 4 đoạn tài liệu và metadata nguồn phù hợp nhất để đưa vào context cho LLM.

1.5 Tính năng nổi bật 2: Hỗ trợ đa ngôn ngữ đầu vào (Language Detection & Translation)

Hệ thống tự động xử lý ngôn ngữ đầu vào mà không cần người dùng khai báo, qua đó giao tiếp nhuần nhuyễn với cả người dùng Việt Nam và quốc tế:

- **Nhận diện ngôn ngữ tự động:** Sử dụng thư viện Lingua (infrastructure/language_detector.py) để phát hiện ngôn ngữ câu hỏi (vi hoặc en).
- **Dịch truy vấn sang tiếng Việt:** Nếu câu hỏi là tiếng Anh, hệ thống dịch sang tiếng Việt (qua mô hình EnViT5 tại infrastructure/translator.py) trước khi tìm kiếm trên cơ sở dữ liệu tiếng Việt.
- **Trả lời đúng ngôn ngữ gốc:** LLM sử dụng prompt tương ứng (MEDICAL_SYSTEM_PROMPT cho tiếng Việt, MEDICAL_SYSTEM_PROMPT_EN cho tiếng Anh) và tạo ra câu trả lời đúng ngôn ngữ của người dùng..
- **Transparent :** Response trả về bao gồm cả trường detected_language và translated_query để minh bạch quá trình xử lý.

1.6 Luồng xử lý RAG Pipeline (End-to-End)

1. Nhận ChatRequest từ API endpoint /api/v1/chat.
2. Nhận diện ngôn ngữ bằng Lingua.
3. Nếu là tiếng Anh: dịch câu hỏi sang tiếng Việt bằng EnViT5.
4. Thực hiện Hybrid Retrieval (Dense + Sparse + RRF Fusion) trên cơ sở dữ liệu.
5. Trích xuất tối đa 4 đoạn tài liệu và metadata nguồn.
6. Chọn prompt phù hợp theo ngôn ngữ (tiếng Việt hoặc tiếng Anh).
7. Gửi ngữ cảnh và câu hỏi đến Gemini / Qwen để sinh câu trả lời.

8. Trả về ChatResponse kèm danh sách nguồn trích dẫn.

1.7 Công nghệ sử dụng

Thành phần	Công nghệ / Mô hình	Mô tả
Framework API	FastAPI (Python)	REST API server nhẹ, hiệu năng cao cho AI/ML serving
Mô hình nhúng văn bản	BAAI/bge-m3 (BGE-M3)	Tạo dense embedding cho tìm kiếm ngữ nghĩa
Vector Database	ChromaDB	Lưu trữ và tìm kiếm vector embedding
Tìm kiếm từ khóa	BM25 (sparse index)	Khớp chính xác thuật ngữ chuyên ngành
Mô hình LLM	Google Gemini 2.5 Flash / Qwen-Max	Sinh câu trả lời y khoa có căn cứ
Mô hình dịch thuật	EnViT5 (API ngoài)	Dịch En->Vi cho truy vấn đầu vào
Nhận diện ngôn ngữ	Lingua (Python library)	Phân biệt tiếng Anh / tiếng Việt

PHAN 2: THAY ĐỔI VÀ MỞ RỘNG TRONG DỰ ÁN CŨ (MedTrans)

2.1 Thay đổi cơ sở dữ liệu

Để hỗ trợ song song hai loại cuộc trò chuyện (Dịch thuật và Chatbot), schema cơ sở dữ liệu được cập nhật thêm cột type vào hai bảng chính:

Script SQL thay đổi:

```
ALTER TABLE conversations ADD COLUMN type ENUM('TRANSLATION', 'CHATBOT') NOT NULL DEFAULT 'TRANSLATION';  
ALTER TABLE messages ADD COLUMN type ENUM('TRANSLATION', 'CHATBOT') NOT NULL DEFAULT 'TRANSLATION';
```

Bảng	Cột thêm	Kiểu dữ liệu	Giá trị mặc định	Ý nghĩa
conversations	type	ENUM('TRANSLATION', 'CHATBOT')	TRANSLATION	Phân loại phiên: Dịch thuật hoặc

Bảng	Cột thêm	Kiểu dữ liệu	Giá trị mặc định	Ý nghĩa
				Chat y tế
messages	type	ENUM('TRANSLATION', 'CHATBOT')	TRANSLATION	Phân loại tin nhắn theo phiên tương ứng

Giá trị mặc định TRANSLATION đảm bảo tương thích ngược (backward compatible) với toàn bộ dữ liệu phiên dịch thuật hiện có.

2.2 Tích hợp API Chatbot vào Backend (Spring Boot)

Backend Spring Boot được bổ sung khả năng gọi API Medical RAG để xử lý các yêu cầu chat y tế:

Thành phần	Thay đổi	Mô tả
AiClient.java	Thêm phương thức gọi /api/v1/chat	Gửi POST request đến http://127.0.0.1:8000/api/v1/chat với payload { message }
ChatController.java	Thêm endpoint xử lý chatbot	Phân biệt yêu cầu theo conversation.type: TRANSLATION gọi /predict, CHATBOT gọi /api/v1/chat
ChatService.java	Thêm logic chatbot	Lưu message, gọi AiClient với endpoint tương ứng, lưu response kèm sources từ Medical RAG
Dtos.java	Thêm ChatbotResponse DTO	Bao gồm answer, detected_language, translated_query, sources[]

2.3 Thay đổi giao diện (Frontend - React)

Giao diện được mở rộng thêm khu vực Chatbot Y tế, hoạt động song song với khu vực Dịch thuật hiện có:

Các thành phần mới / cập nhật:

Component	Loại	Mô tả thay đổi
Sidebar.jsx	Cập nhật	Hiển thị nút tạo cuộc trò chuyện mới với hai lựa chọn: 'Dịch thuật mới' (TRANSLATION) và 'Chat Y tế mới' (CHATBOT). Có thể tách thành hai tab riêng biệt.
ChatArea.jsx	Cập nhật	Phân biệt giao diện theo conversation.type: nếu là CHATBOT, hiển thị thêm khung 'Nguồn tham khảo' (sources) phía dưới mỗi tin nhắn AI.
ChatBotArea.jsx	Thêm mới	Component riêng cho khu vực chat y tế, hiển thị bubble tin nhắn với style phân biệt, hiển thị các nguồn trích dẫn (title, source, url) từ Medical RAG.
Header.jsx	Cập nhật	Ẩn dropdown 'Hướng dịch' (EN->VI / VI->EN) khi conversation.type là CHATBOT, vì hệ thống tự xử lý ngôn ngữ.
api/client.js	Cập nhật	Thêm hàm gọi API chatbot, truyền conversationId và message đến endpoint dịch thuật chatbot của Spring Boot.

Tính năng UX mới trong khu vực Chatbot:

- **Hiển thị nguồn tham khảo:** Mỗi phản hồi AI kèm thêm danh sách nguồn trích dẫn (tên tài liệu, tên file, URL) giúp người dùng kiểm tra thông tin.
- **Hiển thị ngôn ngữ phát hiện:** Thông báo ngôn ngữ được phát hiện (Tiếng Việt / Tiếng Anh) và câu hỏi đã được dịch sang tiếng Việt (nếu có).
- **Latency indicator:** Tương tự khu vực dịch thuật, hiển thị thời gian phản hồi AI kèm biểu tượng sao chép.
- **Placeholder thông minh:** Ô nhập liệu hiển thị gợi ý 'Nhập câu hỏi về sức khỏe...' thay vì 'Nhập văn bản cần dịch...' khi ở khu vực Chatbot.

PHAN 3: ĐƠN GIẢN HÓA BẢO MẬT

3.1 Các thay đổi về bảo mật

Trong quá trình phát triển phiên bản mở rộng này, một số thành phần bảo mật trong dự án gốc MedTrans đã được lược giản hóa có chủ ý. Lý do chính là dự án hiện tại được xây dựng với mục tiêu là một công cụ hỗ trợ cá nhân (personal tool), tập trung vào việc chứng minh tính khả thi của mô hình kết hợp Dịch thuật + Chatbot Y tế, chưa yêu cầu mức độ bảo mật của một sản phẩm thương mại hoàn chỉnh. Việc giữ lại các lớp bảo mật phức tạp ở giai đoạn này sẽ làm tăng đáng kể thời gian phát triển và gây khó khăn trong quá trình debug, kiểm thử tích hợp với các service ngoài.

3.2 Chi tiết các thay đổi

a) Loại bỏ Spring Security

Hệ thống Spring Security (bao gồm JwtAuthFilter, SecurityConfig, phân quyền route) đã được tạm thời loại khỏi dự án. Nguyên nhân cụ thể:

- Dự án hiện tại chỉ có một người dùng duy nhất (bản thân tác giả), chưa có nhu cầu quản lý đa người dùng với phân quyền phức tạp.
- Việc tích hợp thêm Medical RAG (FastAPI) vào luồng xử lý đã tăng đáng kể độ phức tạp; giữ thêm lớp xác thực JWT làm khó khăn quá trình kiểm thử API tích hợp và debug lỗi.
- Hệ thống chạy trên môi trường local (localhost), chưa expose ra internet nên nguy cơ bảo mật thực tế ở giai đoạn này rất thấp.
- Cho phép tập trung nguồn lực vào việc phát triển tính năng AI cốt lõi thay vì cấu hình bảo mật, phù hợp với giai đoạn Proof of Concept (PoC).

b) Bỏ mã hóa mật khẩu (BCrypt Hashing)

Chức năng mã hóa mật khẩu bằng BCryptPasswordEncoder đã được tạm thời loại bỏ, mật khẩu hiện được lưu trực tiếp dưới dạng plain text trong cơ sở dữ liệu local. Lý do:

- Dữ liệu đăng nhập chỉ phục vụ cho việc kiểm thử tính năng, không lưu thông tin cá nhân hay tài sản thực sự có giá trị.
- Việc loại bỏ BCrypt giúp giảm thời gian xử lý login trong quá trình kiểm thử liên tục, tăng tốc độ vòng lặp phát triển (development iteration loop).
- Cơ sở dữ liệu MySQL chỉ chạy trên máy local, không có người ngoài truy cập.